

NOM :

Juin 2022

PRÉNOM :

BLOC :



Examen de Mathématiques 2 :

1^{ère} année Bachelier en Informatique de Gestion

BINV1100 – Mathématiques 2

Date : 15 juin 2022

Durée de l'examen : 3 heures

Nombre de questions : 6

- 1. Sauf avis contraire, toute réponse doit être justifiée.**
2. Si vous n'écrivez pas proprement et lisiblement, votre réponse recevra un zéro.
3. Écrire au crayon est autorisé si le point 2 ci-dessus est respecté.
4. Vous pouvez avoir à votre disposition 10 feuilles recto/verso respectant les conditions suivantes : vos nom et prénom doivent être indiqués, les feuilles doivent être manuscrites, reliées sur toute la longueur de manière à ne pas pouvoir en détacher sans l'arracher et le contenu ne fait pas l'objet de miniaturisation.
5. Pour les questions sur machine, vous devez travailler **sur le U** : . En effet, si vous travaillez ailleurs vos fichiers seront perdus.
6. Les points communiqués en regard des questions sont indicatifs. Des lacunes graves entraîneront l'échec au présent examen.
- 7. Mettez vos noms et prénoms au début de chaque question !**

Question 1	/15
Question 2	/10
Question 3	/15
Question 4	/15
Question 5	/10
Question 6	/15
TOTAL	/80

Nom :

Prénom :

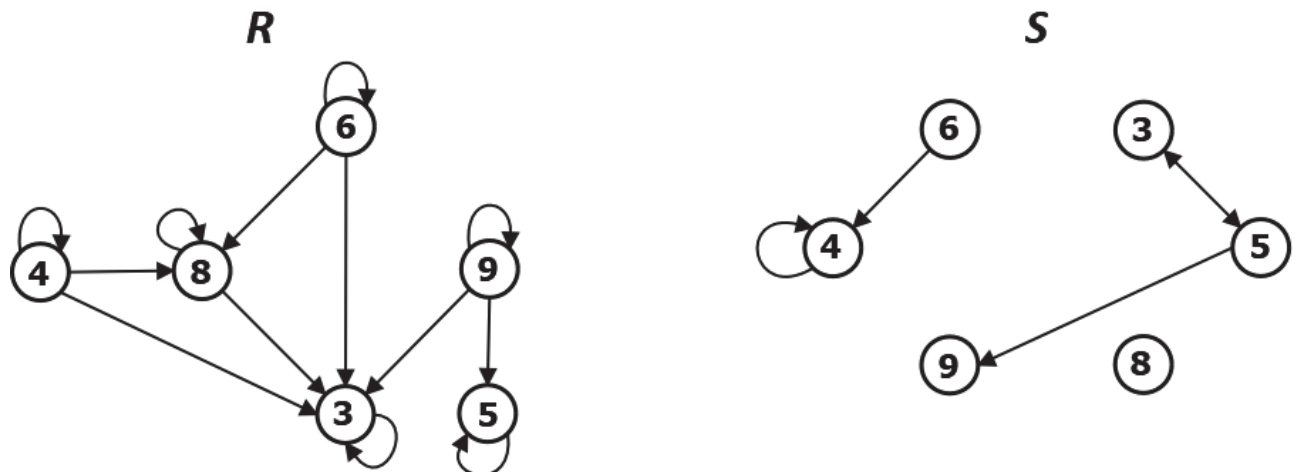
PARTIE I : SUR PAPIER

Nom :

Prénom :

Question 1 (15 pts)

Soit les relations R et S , ci-dessous, sur l'ensemble $E = \{3,4,5,6,8,9\}$.



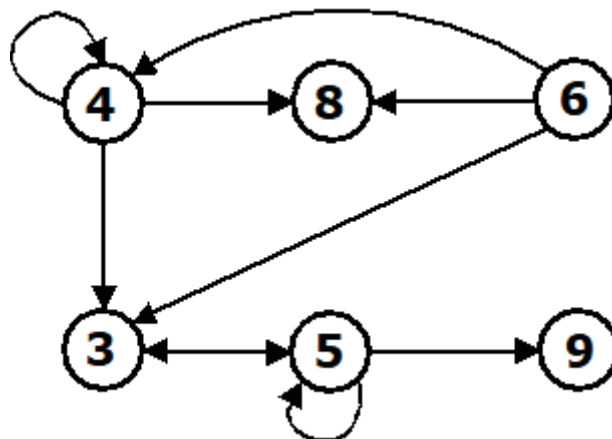
1) a) Donnez le domaine et l'image de la relation S .

$dom(S) = \{3, 4, 5, 6\}$
 $img(S) = \{3, 4, 5, 9\}$

b) La relation S est-elle transitive ? Justifiez.

La relation S n'est pas transitive car $(3, 5) \in S$ et $(5, 9) \in S$ mais $(3, 9) \notin S$

c) Donnez le digraphe de $R \circ S$.

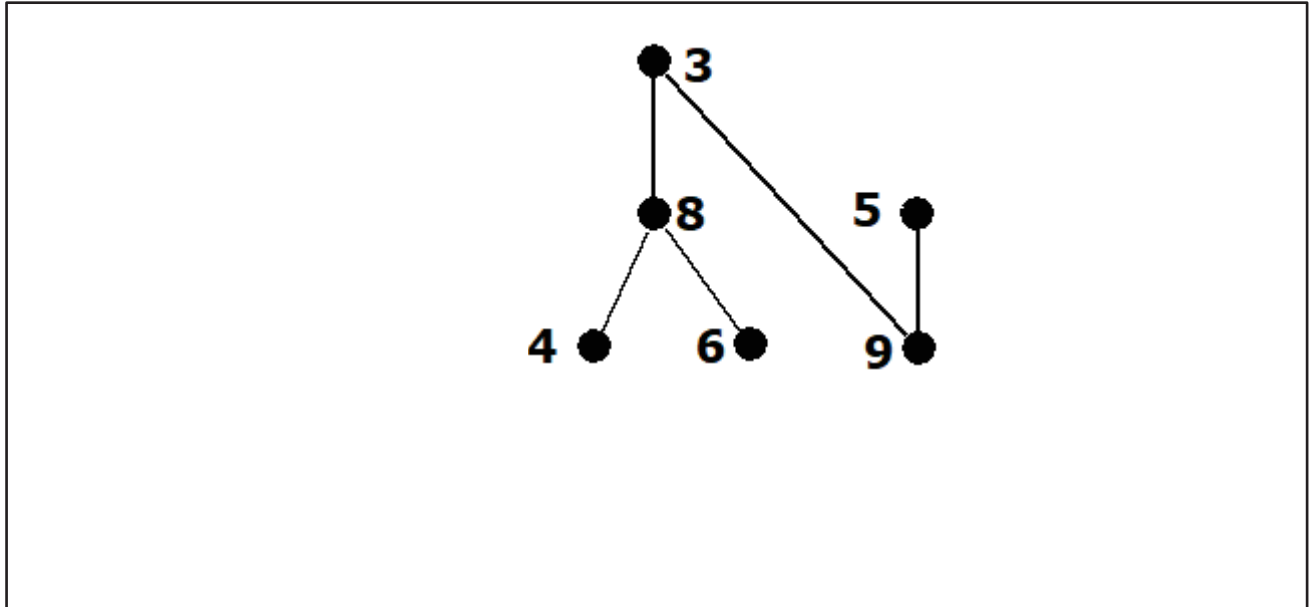


Nom :

Prénom :

2) La relation R est une relation d'ordre.

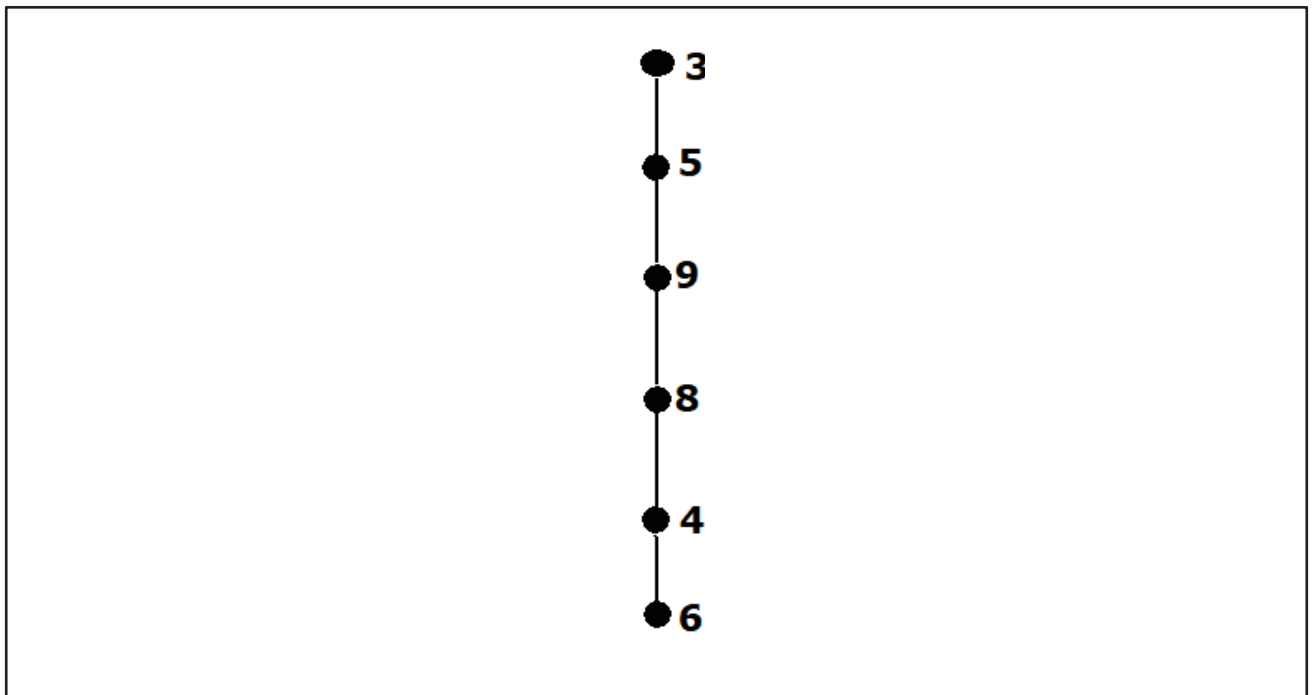
a) Donnez son diagramme de Hasse.



b) Donnez l'ensemble des majorants de $B = \{4,9\}$

$major(B) = \{3\}$

c) Donnez un tri topologique de R



Nom :

Prénom :

Question 2 (10 pts)

Résolvez le système suivant en utilisant la méthode de Gauss.

$$\begin{cases} \frac{4}{3}x_1 - \frac{1}{3}x_2 - \frac{2}{3}x_3 + 2x_4 + \frac{1}{3} = x_1 - \frac{4}{3}x_3 \\ \frac{1}{2}x_1 - 2x_2 + 5x_4 - 4 = x_3 - \frac{1}{2}x_1 + 3 \\ x_1 - 3x_3 - 5 = 2x_2 - 3x_4 + 6 \\ \frac{1}{10}x_2 + \frac{1}{5}x_3 + \frac{2}{5}x_4 = \frac{3}{5} - \frac{1}{5}x_1 + \frac{1}{2}x_2 - x_4 \end{cases}$$

1) Réécriture du Système

$$\Rightarrow \begin{cases} \frac{4}{3}x_1 - \frac{1}{3}x_2 + \frac{4}{3}x_3 - \frac{2}{3}x_3 + 2x_4 - x_1 = -\frac{1}{3} \\ \frac{1}{2}x_1 + \frac{1}{2}x_1 - 2x_2 + 5x_4 - x_3 = 3 + 4 \\ x_1 - 3x_3 - 2x_2 + 3x_4 = 6 + 5 \\ \frac{1}{5}x_1 + \frac{1}{10}x_2 - \frac{1}{2}x_2 + \frac{1}{5}x_3 + \frac{2}{5}x_4 + x_4 = \frac{3}{5} \end{cases} \Rightarrow \begin{cases} \frac{1}{3}x_1 - \frac{1}{3}x_2 + \frac{2}{3}x_3 + 2x_4 = -\frac{1}{3} \\ x_1 - 2x_2 - x_3 + 5x_4 = 7 \\ x_1 - 2x_2 - 3x_3 + 3x_4 = 11 \\ \frac{1}{5}x_1 - \frac{2}{5}x_2 + \frac{1}{5}x_3 + \frac{7}{5}x_4 = \frac{3}{5} \end{cases}$$

2) Echelonnement de la matrice

$$\begin{pmatrix} 1 & -1 & 2 & 6 & -1 \\ 3 & -3 & 3 & 2 & -3 \\ 1 & -2 & -1 & 5 & 7 \\ 1 & -2 & -3 & 3 & 11 \\ 1 & 2 & 1 & 7 & 3 \\ 5 & -5 & 5 & 5 & 5 \end{pmatrix} \xrightarrow[l_4 \leftarrow 5 \cdot l_4]{l_1 \leftarrow 3 \cdot l_1} \begin{pmatrix} 1 & -1 & 2 & 6 & -1 \\ 1 & -2 & -1 & 5 & 7 \\ 1 & -2 & -3 & 3 & 11 \\ 1 & -2 & 1 & 7 & 3 \end{pmatrix}$$

$$\xrightarrow[l_4 \leftarrow l_4 - l_1]{l_2 \leftarrow l_2 - l_1, l_3 \leftarrow l_3 - l_1} \begin{pmatrix} 1 & -1 & 2 & 6 & -1 \\ 0 & -1 & -3 & -1 & 8 \\ 0 & -1 & -5 & -3 & 12 \\ 0 & -1 & -1 & 1 & 4 \end{pmatrix}$$

$$\xrightarrow[l_4 \leftarrow l_4 - l_2]{l_3 \leftarrow l_3 - l_2} \begin{pmatrix} 1 & -1 & 2 & 6 & -1 \\ 0 & -1 & -3 & -1 & 8 \\ 0 & 0 & -2 & -2 & 4 \\ 0 & 0 & 2 & 2 & -4 \end{pmatrix}$$

$$\xrightarrow{l_4 \leftarrow l_4 + l_3} \begin{pmatrix} 1 & -1 & 2 & 6 & -1 \\ 0 & -1 & -3 & -1 & 8 \\ 0 & 0 & -2 & -2 & 4 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

3) Résolution du système

$$\begin{cases} x_1 - x_2 + 2x_3 + 6x_4 = -1 \\ -x_2 - 3x_3 - x_4 = 8 \\ -2x_3 - 2x_4 = 4 \\ 0 = 0 \end{cases} \Rightarrow \begin{cases} x_1 = x_2 - 2x_3 - 6x_4 - 1 \\ -x_2 = 3x_3 + x_4 + 8 \\ -2x_3 - 2x_4 = 4 \\ 0 = 0 \end{cases}$$

$$\begin{cases} x_1 = x_2 - 2x_3 - 6x_4 - 1 \\ x_2 = -3x_3 - x_4 - 8 \\ x_3 = -x_4 - 2 \end{cases} \Rightarrow \begin{cases} x_1 = x_2 - 2(-x_4 - 2) - 6x_4 - 1 \\ x_2 = -3(-x_4 - 2) - x_4 - 8 \\ x_3 = -x_4 - 2 \end{cases}$$

$$\Rightarrow \begin{cases} x_1 = x_2 - 4x_4 + 3 \\ x_2 = 2x_4 - 2 \\ x_3 = -x_4 - 2 \end{cases} \Rightarrow \begin{cases} x_1 = 2x_4 - 2 - 4x_4 + 3 \\ x_2 = 2x_4 - 2 \\ x_3 = -x_4 - 2 \end{cases}$$

$$\Rightarrow \begin{cases} x_1 = -2x_4 + 1 \\ x_2 = 2x_4 - 2 \\ x_3 = -x_4 - 2 \end{cases}$$

$$\text{Donc } S = \{(-2x_4 + 1, 2x_4 - 2, -x_4 - 2, x_4) \mid x_4 \in R\}$$

Nom :

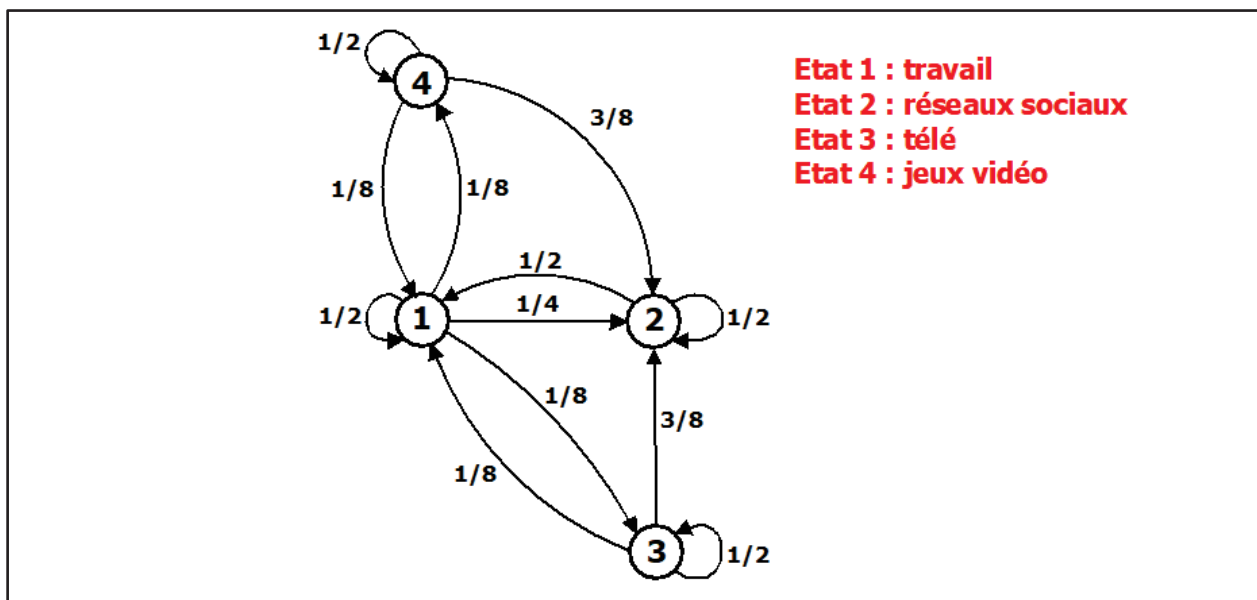
Prénom :

Question 3 (15 pts)

Pendant le blocus, un étudiant est susceptible de changer d'activité toutes les 5 minutes :

- Après 5 minutes d'études, il a une chance sur 4 de commencer à consulter ses réseaux sociaux mais aussi une chance sur 8 de se mettre à regarder la télé et une chance sur 8 de se mettre à jouer à des jeux vidéo. Sinon il continue d'étudier.
- Après avoir consulté ses réseaux, il a alors une chance sur deux de recommencer à étudier et une chance sur deux de poursuivre sa consultation.
- S'il a regardé la télé il a une chance sur 8 de se remettre à étudier après 5 minutes et 3 chances sur 8 de finir par consulter ses réseaux sociaux. Sinon, il continue de regarder la télé.
- S'il a joué à des jeux vidéo, il a une chance sur 8 de se remettre à étudier après 5 minutes et 3 chances sur 8 de finir par consulter ses réseaux sociaux. Sinon, il continue à jouer à des jeux vidéo.

1. Dessinez le graphe du processus de Markov associé.



2. Donnez la matrice de transition du processus

$$\begin{pmatrix} 1/2 & 1/2 & 1/8 & 1/8 \\ 1/4 & 1/2 & 3/8 & 3/8 \\ 1/8 & 0 & 1/2 & 0 \\ 1/8 & 0 & 0 & 1/2 \end{pmatrix}$$

Nom :

Prénom :

3. Donnez les classes de communication, la période de chaque classe et la nature de chaque état.

Une seule classe de communication {1,2,3,4 dont les états sont récurrents de période 1

4. Le processus admet-il une distribution stable ? Est-elle unique ? **Justifiez sans la calculer !**

Le processus admet une distribution stable unique car il est irréductible puis qu'il n'a qu'une classe de communication.

5. Décrivez (équation & brève explication) les deux méthodes permettant de trouver une distribution stable du processus si elle existe.

Méthode 1 :

Résolution du système $M \cdot d = d$ avec la somme des éléments de d égale à 1.

Méthode 2 :

Choisir une distribution de départ d et faire $d = M \cdot d$ jusqu'à convergence. La convergence étant assurée quelque soit la distribution de départ par le théorème de convergence car le système est irréductible (une seule classe de communication) et apériodique (les états ont une période de 1).

Nom :

Prénom :

Question 4 (15 pts)

- 1) a) Soit L le langage, sur l'alphabet $\Sigma = \{a, b, c\}$, formé de tous les mots comportant au minimum 2 lettres, comprenant un nombre impair de b et dans lesquels il n'y a jamais deux a consécutifs. Donnez une grammaire régulière engendrant ce langage. Précisez bien à quel « état » correspond chaque symbole non terminal !

<S> : 0 lettre

<A> : une lettre a

** : une lettre b**

<C> : une lettre c

<D> : au moins deux lettres, nombre pair de b et termine par a

<E> : au moins deux lettres, nombre pair de b et ne termine pas par a

<F> : au moins deux lettres, nombre impair de b et termine par a

<G> : au moins deux lettres, nombre impair de b et ne termine pas par a

Grammaire :

<S> $\rightarrow$ a<A> | b | c<C>

<A> $\rightarrow$ b<G> | c<E>

** $\rightarrow$ a<F> | b<E> | c<G>**

<C> $\rightarrow$ a<D> | b<G> | c<E>

<D> $\rightarrow$ b<G> | c<E>

<E> $\rightarrow$ a<D> | b<G> | c<E>

<F> $\rightarrow$ ϵ | b<E> | c<G>

<G> $\rightarrow$ ϵ | a<F> | b<E> | c<G>

- b) Soit L le langage, sur l'alphabet $\Sigma = \{a, b, c\}$, formé de tous les mots commençant par b ou c et dans lesquels au moins une des lettres ne s'y trouvent pas. Donnez un automate de Moore reconnaissant ce langage. Précisez bien à quel mot correspond chaque état !

Etat 1 : mot vide

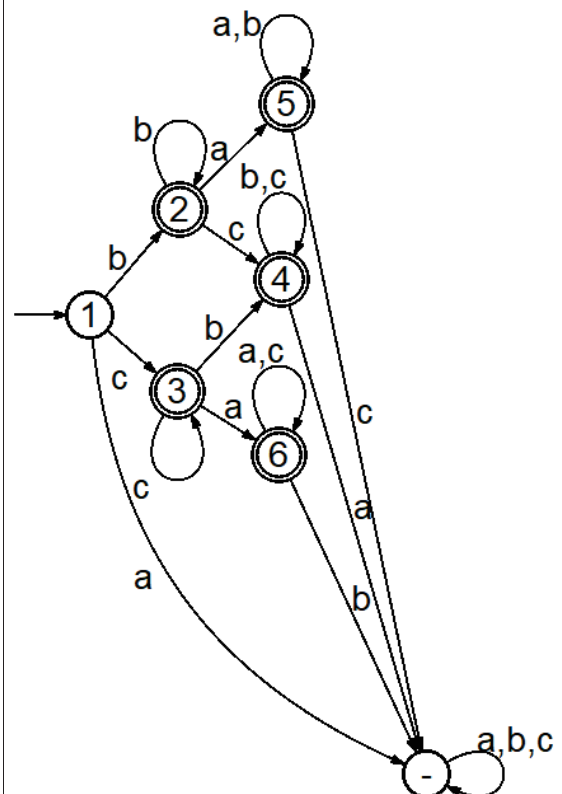
Etat 2 : ne contient que des b

Etat 3 : ne contient que des c

Etat 4 : contient au moins un b et un c et pas de a

Etat 5 : contient au moins un b et un a et pas de c

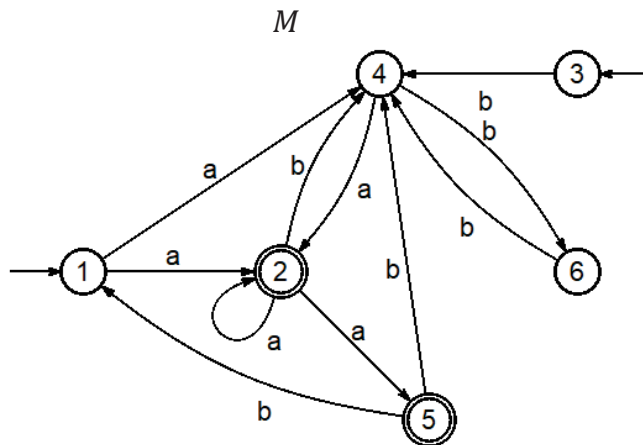
Etat 6 : contient au moins un c et un a et pas de b



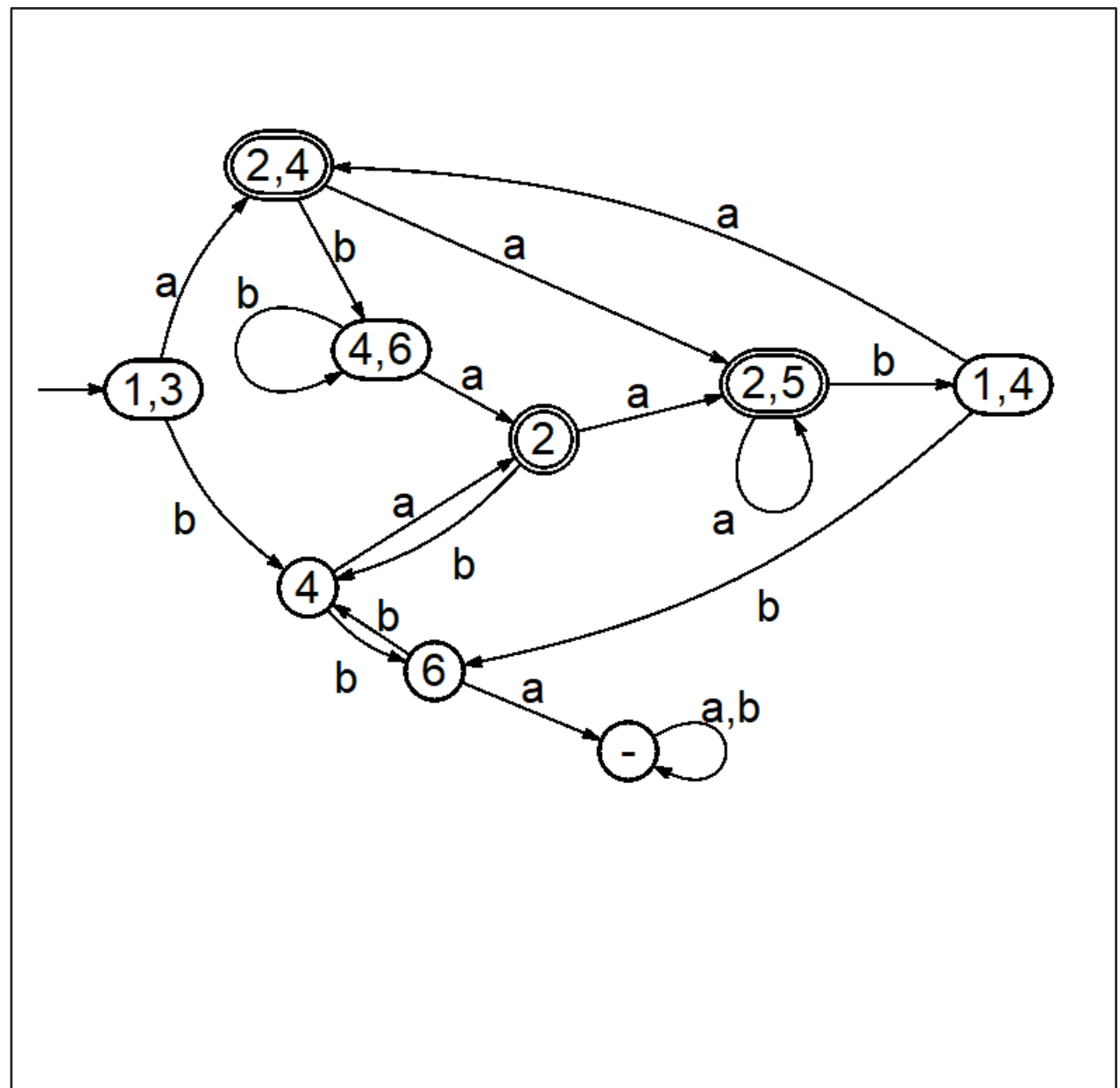
Nom :

Prénom :

2) Soit L le langage défini sur $\Sigma = \{a, b\}$ reconnu par le NFA M .



Utilisez la subset construction afin d'obtenir un automate de Moore reconnaissant le langage L :



Nom :

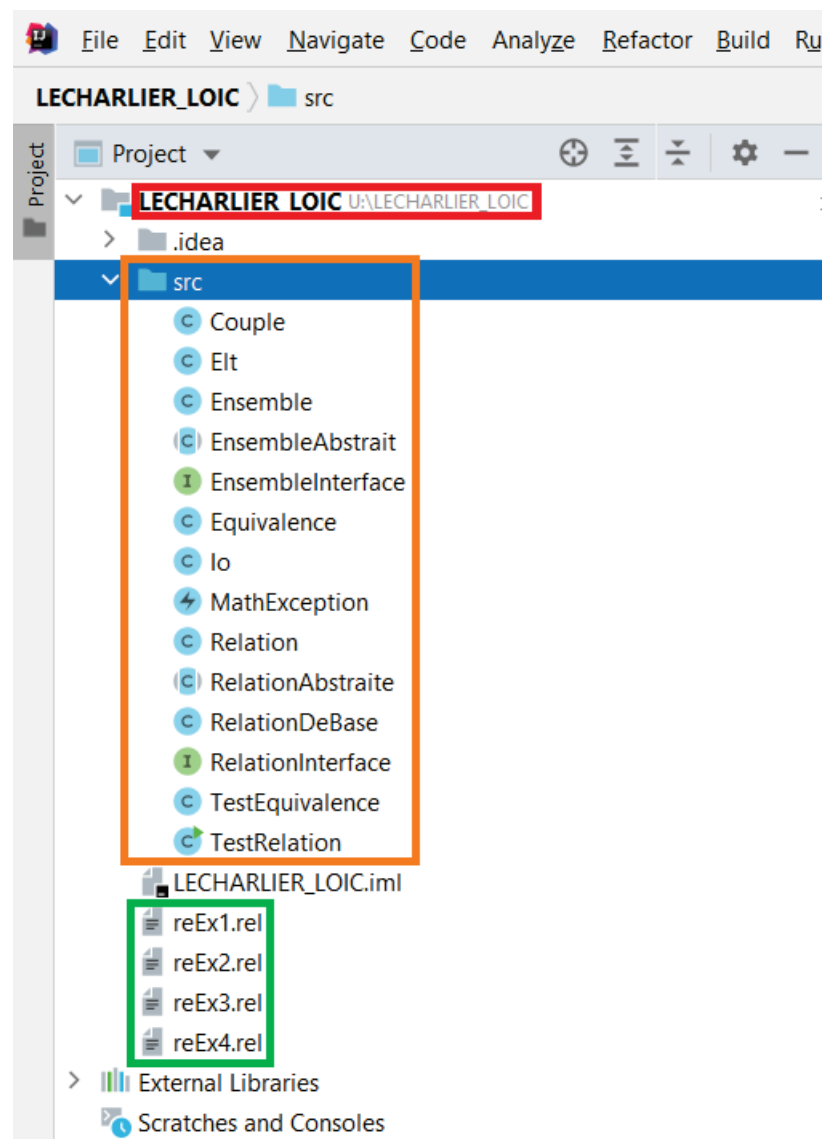
Prénom :

PARTIE II : IMPLÉMENTATION

Dans cette partie nous allons vous demander d'implémenter des méthodes dans plusieurs classes Java.

Pour ce faire :

- 1) Ouvrez IntelliJ
- 2) Créez, **sur le U** , un projet NOM_PRENOM (**avec vos nom et prénom !**)
- 3) Les classes données se trouvent dans le répertoire « Classes Java ». Faites un copier-coller de celles-ci dans le répertoire « src » de votre projet IntelliJ. Voici ce que vous devriez obtenir :



- 4) Les fichiers .rel nécessaires aux tests doivent être placés à la racine du projet comme sur l'image ci-dessous.

Les questions 5 et 6 ci-après vous expliquerons ce que vous devez implémenter.

Nom :

Prénom :

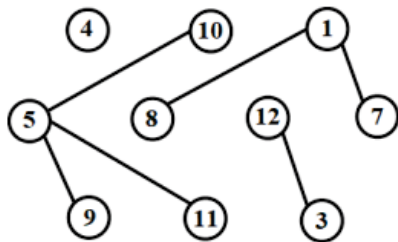
Question 5 (10 pts)

Les instances de la classe *Equivalence* sont des relations d'équivalence sur une partie de l'univers. La partie de l'univers sur laquelle est définie la relation d'équivalence est stockée dans un *EnsembleAbstrait* (sousJac).

On a choisi de représenter une équivalence en choisissant pour chaque classe un représentant. Afin de réaliser cela, la classe *Equivalence* garde un tableau d'*Elt* (tabRep) dans lequel on stocke à l'indice correspondant à l'*Elt* le représentant de sa classe.

Exemple :

Soit $\sim$ une relation d'équivalence sur $\{1,3,4,5,7,8,9,10,11,12\}$.



Cette relation $\sim$ a 4 classes d'équivalence : $\{4\}$, $\{5,9,10,11\}$, $\{1,7,8\}$ et $\{3,12\}$

On choisit un représentant par classe.

Par exemple : $\{\underline{4}\}$, $\{5,\underline{9},10,11\}$, $\{1,\underline{7},8\}$ et $\{\underline{3},12\}$

sousJac = $\{1,3,4,5,7,8,9,10,11,12\}$

tabRep =

\	7	\	3	4	9	\	7	7	9	9	9	3	\		\
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...	MAX

Remarque :

Le contenu de la table dépend du choix des représentants

Si, dans l'exemple : $\{\underline{4}\}$, $\{\underline{5},9,10,11\}$, $\{\underline{1},7,8\}$ et $\{\underline{3},12\}$

sousJac = $\{1,3,4,5,7,8,9,10,11,12\}$

tabRep =

\	1	\	3	4	5	\	1	1	5	5	5	3	\		\
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...	MAX

Dans la classe *Equivalence*, complétez le constructeur qui reçoit en paramètre une table avec les classes d'équivalence.

Testez-le avec la classe *TestEquivalence*.

```

1  public class Equivalence{
2
3      /** Valeur numerique de MAXELT */
4      private static final int MAX = Elt.MAXELT.val();
5
6      private EnsembleAbstrait sousJac; // ensemble sous-jacent
7      private Elt[] tabRep; // tableau des representants
8      // Ne pas ajouter d'attribut
9
10     // si possible, construit une relation d'equivalence
11     // sinon, lance une IllegalArgumentException
12     public Equivalence(EnsembleAbstrait[] classesEquivalence) {
13         //TODO
14         if (classesEquivalence==null)
15             throw new IllegalArgumentException("tableau null") ;
16         sousJac = new Ensemble();
17         tabRep = new Elt[MAX+1];
18         for (int i=0 ; i< classesEquivalence.length ;i++) {
19             if (classesEquivalence[i]==null || classesEquivalence[i].cardinal()==0) {
20                 throw new IllegalArgumentException("Classe null ou vide") ;
21             }
22             Elt rep = classesEquivalence[i].unElement() ;
23             for (Elt e:classesEquivalence[i]) {
24                 if (sousJac.contient(e))
25                     throw new IllegalArgumentException("un élément est dans deux
26                     classes différentes") ;
27                 sousJac.ajouter(e) ;
28                 tabRep[e.val()]=rep ;
29             }
30         }
31     }
32 }
33

```

Nom :

Prénom :

Question 6 (15 pts)

On vous demande d'implémenter la somme disjointe de deux ensembles.

Si A et B sont deux ensembles alors leur somme disjointe, notée $A + B$, est

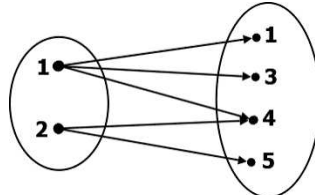
$$A + B = \{(1, a) \mid a \in A\} \cup \{(2, b) \mid b \in B\}$$

Autrement dit, la somme disjointe de A et B est une relation qui associe 1 à tous les éléments de A et 2 à tous les éléments de B .

Exemples :

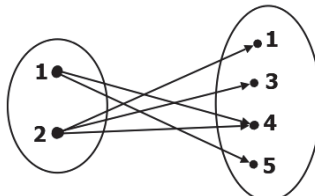
- 1) Si $A = \{1,3,4\}$ et $B = \{4,5\}$, alors $A + B = \{(1,1), (1,3), (1,4), (2,4), (2,5)\}$.

Voici une représentation sagittale de la relation obtenue :



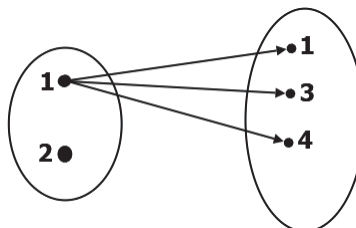
- 2) Si $A = \{4,5\}$ et $B = \{1,3,4\}$, alors $A + B = \{(1,4), (1,5), (2,1), (2,3), (2,4)\}$.

Voici une représentation sagittale de la relation obtenue



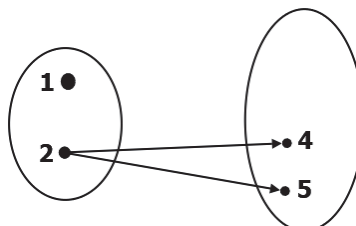
- 3) Si $A = \{1,3,4\}$ et $B = \{\}$, alors $A + B = \{(1,1), (1,3), (1,4)\}$.

Voici une représentation sagittale de la relation obtenue



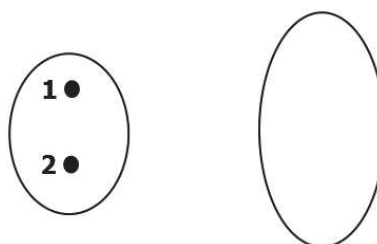
- 4) Si $A = \{\}$ et $B = \{4,5\}$, alors $A + B = \{(2,4), (2,5)\}$.

Voici une représentation sagittale de la relation obtenue



- 5) Si $A = \{\}$ et $B = \{\}$, alors $A + B = \{\}$.

Voici une représentation sagittale de la relation obtenue



```

1 public abstract class EnsembleAbstrait implements EnsembleInterface {
2
3     /**
4     * Renvoie une Relation représentant this + a
5     *
6     * @parameter a : la relation a "sommer" avec this
7     * @return une Relation représentant la somme disjointe de this et a
8     * @throws IllegalArgumentException en cas de paramètre invalide
9     */
10    public Relation sommeDisjointe(EnsembleAbstrait a) {
11        //TODO
12        if (a==null)
13            throw new IllegalArgumentException() ;
14        EnsembleAbstrait depart = new Ensemble();
15        EnsembleAbstrait arrivee = new Ensemble() ;
16        depart.ajouter(new Elt(1));
17        depart.ajouter(new Elt(2));
18        for (Elt e:this) {
19            arrivee.ajouter(e);
20        }
21        for (Elt e:a) {
22            arrivee.ajouter(e);
23        }
24        Relation sommeD = new Relation(depart,arrivee);
25        for (Elt e:this) {
26            sommeD.ajouter(new Elt(1),e);
27        }
28        for (Elt e:a) {
29            sommeD.ajouter(new Elt(2),e);
30        }
31        return sommeD ;
32    }
33
34    public boolean inclusDans (EnsembleAbstrait e) {
35        if (e==null)
36            return false ;
37        for (Elt el:this) {
38            if (!e.contient(el))
39                return false ;
40        }
41        return true ;
42    }
43
44    // renvoie true ssi this est égal à o
45    public final boolean equals(Object o) {
46        if (o == null)
47            return false;
48        if (o == this)
49            return true;
50        if (!(o instanceof EnsembleAbstrait))
51            return false ;
52        //TODO
53        EnsembleAbstrait e = (EnsembleAbstrait) o ;
54        return this.inclusDans(e) && e.inclusDans(this) ;
55    }
56
57    @Override
58    public final int hashCode() {
59        int result = 1;
60        int prime = 31;
61        for (int i = 1; i <= MAX; i++) {
62            Elt ei = new Elt(i);
63            if (this.contient(ei))
64                result = result * prime + ei.hashCode();
65        }
66        return result;
67    }
68
69    @Override
70    public EnsembleAbstrait clone() {
71        try {
72            return (EnsembleAbstrait) super.clone();
73        } catch (CloneNotSupportedException e) {

```